

1. Consider the code below. You are supposed to update the value of variable 'a' without directly modifying it. That is, you have to modify it through one of the other two variables: b and p. Will the following code modify a? If not, correct it. You are not allowed to introduce new variables.

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
```

```
void main()
{
    int a, b, *p;

    a = 10;
    b = a;

    p = &a;

    b++;

    printf("a = %d\n", a);

}
```

2. Consider a **double** type array (a) of five elements. Let p be a pointer pointing to array a. Using only p (without using a directly), find if a specific element E is present in array a; if yes, increment E in a by 1. Then, print the updated array using p. E.g., if a is {1.5, 2.4, 5.8, 11.35, 4.4} and E is 2.4, then the modified version of a (after the increment) will be {1.5, **3.4**, 5.8, 11.35, 4.4}.
3. Spot errors in the program that is supposed to print a modified string in the main through change() and correct it:

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
```

```
char* change(char *s)//changes the first character of the argument string
{
```

```

char s_out[10];

s_out = (char*)malloc(sizeof(char)*(strlen(s)+1));

strcpy(s_out, s);

s_out[0]--;

return s_out;

}

```

```

void main()
{
    char s[] = "hello", *s_out;

    s_out = change(s);

    printf("%s %s\n", s, s_out);

}

```

4. Spot the issues in the program that is supposed to print an incremented version of a variable:

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>

```

```

void main()
{
    int *a;

```

```

*a = 10;

*a++;

printf("%d\n", *a);

```

5. Consider a database of student records (stored in a structure): name (a multi-word string), id (integer), marks in three subjects as a static array of floating point numbers, average marks (float), and grade letter (string).
Take name, id, and marks in three subjects as input from the user in a function `inputStudentData()` that takes a pointer to an array of structures as an argument. Then pass this pointer as an argument to the function `calculateStatistics()` to compute the average marks and grade letter (for the latter use IISER-K standard grade boundaries) in the structure.
Display all information for all the students by calling a function `printallStudents()` that takes the same pointer as an argument.
6. Write a program to dynamically allocate an array of strings, where you do not know the number of strings (and each string or the length) in advance.
7. Consider a database of students allocated dynamically. Each student has an id and some courses, stored as a structure element. Here, both id (as a pointer to char) and courses (as a pointer-to-pointer to char) are allocated dynamically, where the courses (and the number of such courses) can be different for different students. Also, you don't know in advance the number of students, which you need to ask the user. Write a program that takes the aforementioned information from the user for some students and prints them all via a display function `print_stu()` that takes the database (as a pointer to a structure) as arguments.
8. The following code is supposed to print all the elements in the function in `f()`. However, the code is wrong, and please correct it. Note that the number of elements in the array has to be computed automatically in `main()`.
9. Use Valgrind to check for memory leaks in the following code:

```

#include<stdio.h>
#include<string.h>
#include<stdlib.h>

void main()

```

```

{
    int *a;

    a = (int*)malloc(5*sizeof(int));

    free(a);

    printf("%d\n", a[0]);

}

```

10. Consider an acid test situation where a stranger is asked to solve some tasks to check whether the person is an agent of a secret detective agency named KHOPAT. The stranger is asked to solve three tasks named *epanq*, *opanq*, and *jhopanq* – one after the other. If any of the tasks are performed wrongly, the person is declared “Trespasser” immediately (without the need to perform the remaining tasks, if any), followed by serious consequences, while if all the tasks are performed correctly, the person is warmly welcomed.

To make the final decision (Trespasser or not), design a C function named *ghochanq()* that takes pointers to functions *epanq*, *opanq*, and *jhopanq* as arguments. Now, given two randomly generated integers *x* and *y*, *epanq*, *opanq*, and *jhopanq* functions are defined as below:

$n1 = \text{epanq}(x, y) = 2x + y$, $n2 = \text{opanq}(x, y) = 3y + x$ and $n3 = \text{jhopanq}(n1, n2)$ is a two-digit integer composed of the last two digits of the integers *n1* and *n2* in this order.

E.g if $x = 5$, $y = 6$, $n1 = 16$, $n2 = 23$ and $n3 = 63$.

Furthermore, you have to generate *x* and *y* randomly within any range of your choice. In C, the following piece of code generates a random integer within a range [min, max]:

```

#include<stdlib.h>
#include <time.h>

...

void main()
{

```

```
int max = 10, min = 5, rd_num;
```

```
    srand(time(0)); //used to initialize or set the seed for the 'rand()' function, allowing us to  
    generate different sequences of random numbers.
```

```
        rd_num = rand() % (max - min + 1) + min; // rd_num is the random number  
        between min and max
```

```
    }
```